

Reporte de la práctica: Efectos de iluminación con 10 LEDs (Parpadeo independiente, Knight Rider y Cometa)

1. Datos generales

Materia	Desarrollo Sustentable
Práctica	Efectos de iluminación con 10 LEDs controlados por botón
Placa utilizada	Arduino UNO R4 WiFi
Entorno de desarrollo	Arduino IDE
Componentes	Arduino UNO R4 WiFi (utilizando su botón de reset integrado como entrada en el pin D2), 10 LEDs de colores variados (rojo, blanco, azul, verde y amarillo), 10 resistencias de 220 Ω , protoboard, cables de conexión y computadora

2. Objetivo

Programar y poner a prueba tres efectos de iluminación distintos sobre un arreglo de 10 LEDs, alternando entre ellos mediante un botón, aplicando lectura digital con antirrebote (debounce), control de tiempo no bloqueante con millis() y modulación de brillo mediante PWM por software.

3. Diagrama del circuito (conexiones)

Componente	Conexión
LED 1 (ánodo)	Pin digital 13 del Arduino
LED 2 (ánodo)	Pin digital 12 del Arduino
LED 3 (ánodo)	Pin digital 11 del Arduino
LED 4 (ánodo)	Pin digital 10 del Arduino
LED 5 (ánodo)	Pin digital 9 del Arduino
LED 6 (ánodo)	Pin digital 8 del Arduino
LED 7 (ánodo)	Pin digital 7 del Arduino
LED 8 (ánodo)	Pin digital 6 del Arduino
LED 9 (ánodo)	Pin digital 5 del Arduino
LED 10 (ánodo)	Pin digital 4 del Arduino
Cátodo de cada LED	Terminal de su resistencia de 220 Ω correspondiente (en la misma fila del protoboard)
Otra terminal de cada resistencia	GND del Arduino (riel negativo del protoboard)
Botón	Botón de reset integrado en la propia placa Arduino, reutilizado como entrada digital en el pin D2 (no es un componente externo)

Nota: los 10 LEDs son de distintos colores (rojo, blanco, azul, verde y amarillo), distribuidos entre los pines de forma aleatoria, sin un orden de color fijo.

4. Descripción del funcionamiento

4.1 Botón y cambio de efecto

Para esta práctica se reutilizó el botón de reset que trae integrado el propio Arduino (en lugar de un botón externo), conectando su salida al pin digital 2 configurado como INPUT_PULLUP. El programa detecta el flanco de bajada (botón presionado) aplicando un antirrebote (debounce) de 50 ms mediante la función revisarBoton(). Cada vez que se detecta una pulsación válida, se avanza al siguiente efecto de forma cíclica (0 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 0) y se reinician las variables de todos los efectos para que cada uno inicie siempre desde un estado limpio.

4.2 Efecto 1: Parpadeo independiente

Cada uno de los 10 LEDs parpadea con un intervalo de tiempo distinto (200, 300, 400... hasta 1100 ms), de forma independiente entre sí. Esto se logra usando millis() en lugar de delay(), de manera que el estado de cada LED se actualiza únicamente cuando ha transcurrido su propio intervalo, sin detener la ejecución del resto del programa.

4.3 Efecto 2: Knight Rider

Un solo LED permanece encendido a la vez y se desplaza de un extremo al otro del arreglo, invirtiendo su dirección al llegar a los pines de los extremos, con un intervalo fijo de 150 ms. El efecto simula el barrido luminoso del auto de la serie "El Auto Fantástico" (Knight Rider).

4.4 Efecto 3: Cometa con chispas

Este efecto simula una estela luminosa (cometa) que recorre el arreglo de LEDs: el LED en la posición actual se enciende al máximo brillo (255) y los demás se van apagando gradualmente (desvanecimiento de 45 unidades por ciclo), generando el efecto de cola. Adicionalmente, de forma aleatoria puede encenderse una "chispa" en un LED distinto al de la posición actual. El brillo de cada LED se controla mediante PWM por software, comparando un contador cíclico (cicloPWM) contra el nivel de brillo asignado a cada LED.

5. Código fuente

```
// Múltiples efectos de LEDs, cambian con un botón
// Efectos: 1) Parpadeo independiente  2) Knight Rider  3) Cometa con chispas
// Autor: Código de ejemplo modificado

const int cantidadPines = 10;
const int pinLED[cantidadPines] = {13, 12, 11, 10, 9, 8, 7, 6, 5, 4};
const int pinBoton = 2;

const int cantidadEfectos = 3;
int efectoActual = 0;

// --- Variables para debounce del botón ---
bool estadoBotonAnterior = HIGH;
unsigned long ultimoRebote = 0;
const unsigned long tiempoRebote = 50;

// --- Variables del Efecto 1: Parpadeo independiente ---
const long intervaloParpadeo[cantidadPines] = {200, 300, 400, 500, 600, 700, 800, 900, 1000, 1100};
unsigned long ultimoCambioParpadeo[cantidadPines] = {0};
bool estadoLEDParpadeo[cantidadPines] = {false};

// --- Variables del Efecto 2: Knight Rider ---
const long intervaloKnightRider = 150;
unsigned long ultimoCambioKnightRider = 0;
int posicionKnightRider = 0;
int direccionKnightRider = 1;

// --- Variables del Efecto 3: Cometa con chispas ---
int brillo[cantidadPines] = {0};
const int desvanecimiento = 45;
const long intervaloCometa = 80;
unsigned long ultimoCambioCometa = 0;
```

```

int posicionCometa = 0;
int direccionCometa = 1;
unsigned long cicloPWM = 0;
const int resolucionPWM = 256;

void setup() {
  for (int i = 0; i < cantidadPines; i++) {
    pinMode(pinLED[i], OUTPUT);
  }
  pinMode(pinBoton, INPUT_PULLUP);
  randomSeed(analogRead(A0));
}

void loop() {
  revisarBoton();

  switch (efectoActual) {
    case 0: efectoParpadeoIndependiente(); break;
    case 1: efectoKnightRider(); break;
    case 2: efectoCometa(); break;
  }
}

// ===== BOTÓN =====
void revisarBoton() {
  bool lectura = digitalRead(pinBoton);

  // Detecta flanco de bajada (botón presionado) con debounce
  if (lectura == LOW && estadoBotonAnterior == HIGH &&
      (millis() - ultimoRebote) > tiempoRebote) {
    ultimoRebote = millis();
    cambiarEfecto();
  }
  estadoBotonAnterior = lectura;
}

void cambiarEfecto() {
  efectoActual = (efectoActual + 1) % cantidadEfectos;
  apagarTodos();
  reiniciarVariablesEfecto();
}

void apagarTodos() {
  for (int i = 0; i < cantidadPines; i++) {
    digitalWrite(pinLED[i], LOW);
  }
}

// Reinicia variables para que cada efecto siempre empiece "limpio"
void reiniciarVariablesEfecto() {
  for (int i = 0; i < cantidadPines; i++) {
    estadoLEDParpadeo[i] = false;
    ultimoCambioParpadeo[i] = 0;
    brillo[i] = 0;
  }
  posicionKnightRider = 0;
}

```

```

direccionKnightRider = 1;
posicionCometa = 0;
direccionCometa = 1;
}

// ===== EFECTO 1: Parpadeo independiente =====
void efectoParpadeoIndependiente() {
    unsigned long tiempoActual = millis();
    for (int i = 0; i < cantidadPines; i++) {
        if (tiempoActual - ultimoCambioParpadeo[i] >= intervaloParpadeo[i]) {
            ultimoCambioParpadeo[i] = tiempoActual;
            estadoLEDParpadeo[i] = !estadoLEDParpadeo[i];
            digitalWrite(pinLED[i], estadoLEDParpadeo[i] ? HIGH : LOW);
        }
    }
}

// ===== EFECTO 2: Knight Rider =====
void efectoKnightRider() {
    unsigned long tiempoActual = millis();
    if (tiempoActual - ultimoCambioKnightRider >= intervaloKnightRider) {
        ultimoCambioKnightRider = tiempoActual;

        apagarTodos();
        digitalWrite(pinLED[posicionKnightRider], HIGH);

        posicionKnightRider += direccionKnightRider;
        if (posicionKnightRider == cantidadPines - 1 || posicionKnightRider == 0) {
            direccionKnightRider = -direccionKnightRider;
        }
    }
}

// ===== EFECTO 3: Cometa con chispas =====
void efectoCometa() {
    unsigned long tiempoActual = millis();

    if (tiempoActual - ultimoCambioCometa >= intervaloCometa) {
        ultimoCambioCometa = tiempoActual;

        for (int i = 0; i < cantidadPines; i++) {
            brillo[i] = max(0, brillo[i] - desvanecimiento);
        }
        brillo[posicionCometa] = 255;

        if (random(0, 10) == 0) {
            int chispa = random(0, cantidadPines);
            brillo[chispa] = random(80, 180);
        }

        posicionCometa += direccionCometa;
        if (posicionCometa == cantidadPines - 1 || posicionCometa == 0) {
            direccionCometa = -direccionCometa;
        }
    }
}

```

```
// PWM por software para este efecto
cicloPWM = (cicloPWM + 1) % resolucionPWM;
for (int i = 0; i < cantidadPines; i++) {
    digitalWrite(pinLED[i], cicloPWM < brillo[i] ? HIGH : LOW);
}
}
```

6. Conclusión

La práctica permitió aplicar el manejo de múltiples salidas digitales de forma simultánea y no bloqueante mediante `millis()`, a diferencia del uso de `delay()` que detendría todo el programa. También se puso en práctica la lectura de un botón con antirrebote por software y la generación de PWM por software para controlar el brillo de los LEDs sin usar pines PWM dedicados. El resultado fueron tres efectos visuales claramente diferenciados, controlables en tiempo real con una sola entrada digital.